



Instituto Politécnico Nacional

Escuela Superior de Cómputo

ADMINISTRACIÓN DE SERVICIOS EN RED

Actividad:

REPORTE DE PRACTICA 6 GNS3

Alumno:

Sigala Morales Said

2022630413

Grupo: 6CM4

Fecha de realización: 29 de octubre de 2024

Fecha de entrega: 29 noviembre de 2024



1. DEFINICIÓN DEL JUEGO DEL RELOJ EN AJEDREZ. ¡Error! Marcador no definido.
2. REGLAS DE INFERENCIA DEL JUEGO..... ¡Error! Marcador no definido.
3. REGLAS DE INFERENCIA EN SENTENCIAS PROPOSICIONALES. ¡Error! Marcador no definido.



Instituto Politécnico Nacional
Escuela Superior de Cómputo
ADMINISTRACIÓN DE SERVICIOS EN RED



```
root@Python,Go,Perl,PHP-1:~# cat bueno.py
import telnetlib
import time
from flask import Flask, jsonify
import pygal
from datetime import datetime

# Configuración de Flask
app = Flask(__name__)

# Variables globales
traffic_data = [] # Almacena los paquetes de entrada
events = [] # Almacena eventos (linkup, linkdown)
monitoring_active = False

# Configuración del router
HOST = "192.168.3.2" # IP del router
USERNAME = "admin" # Usuario configurado en el router
PASSWORD = "admin" # Contraseña configurada en el router
INTERFACE = "FastEthernet2/0" # Cambia según tu configuración

# Función para conectarse al router y obtener datos de la interfaz
def get_router_data():
    try:
        # Conectar al router
        tn = telnetlib.Telnet(HOST)

        # Ingresar credenciales
        tn.read_until(b"Username: ")
        tn.write(USERNAME.encode('ascii') + b"\n")
        tn.read_until(b"Password: ")
        tn.write(PASSWORD.encode('ascii') + b"\n")

        # Verificar el estado de la interfaz
        tn.write(f"show interface {INTERFACE}".encode('ascii') + b"\n")
        time.sleep(1)
        output = tn.read_very_eager().decode('ascii')

        tn.close()

        # Analizar salida
        packets = 0
        status = "down"
```



```
if "packets input" in output:
    packets = int(output.split("packets input")[0].split()[-1])
if "line protocol is up" in output:
    status = "up"

    return packets, status

except Exception as e:
    print(f"Error fetching data: {e}")
    return None, None

# Función de monitoreo
def monitor_interface():
    global traffic_data, events, monitoring_active
    previous_status = "down"
    while True:
        if monitoring_active:
            packets, status = get_router_data()
            timestamp = datetime.now()

            if packets is not None:
                traffic_data.append((timestamp, packets))
                # Imprimir datos de paquetes monitoreados
                print(f"[{timestamp}] Packets: {packets}")

            if status is not None and status != previous_status:
                events.append((timestamp, f"Link {status}"))
                previous_status = status
                # Imprimir eventos de link up/down
                print(f"[{timestamp}] Event: Link {status}")

            time.sleep(20) # Monitoreo cada 20 segundos
        else:
            time.sleep(1)

# API REST para iniciar el monitoreo
@app.route('/monitor', methods=['POST'])
def start_monitoring():
    global monitoring_active
    monitoring_active = True
    return jsonify({"message": "Monitoring started"}), 200

# API REST para detener el monitoreo
@app.route('/stop_monitor', methods=['POST'])
def stop_monitoring():
    global monitoring_active
```



```
def start_monitoring():
    global monitoring_active
    monitoring_active = True
    return jsonify({"message": "Monitoring started"}), 200

# API REST para detener el monitoreo
@app.route('/stop_monitor', methods=['POST'])
def stop_monitoring():
    global monitoring_active
    monitoring_active = False
    return jsonify({"message": "Monitoring stopped"}), 200

# API REST para generar la gráfica
@app.route('/generate_graph', methods=['GET'])
def generate_graph():
    global traffic_data, events

    if not traffic_data:
        return jsonify({"error": "No data available for graph"}), 400

    times = [data[0].strftime("%H:%M:%S") for data in traffic_data]
    traffic = [data[1] for data in traffic_data]

    line_chart = pygal.Line(x_label_rotation=20, show_minor_x_labels=False)
    line_chart.title = "Interface Traffic and Events"
    line_chart.x_labels = times
    line_chart.add("Traffic (packets)", traffic)

    for event in events:
        line_chart.add(event[1], [{"value": None, "x": event[0].strftime("%H:%M:%S")}])

    chart_file = "traffic_graph.svg"
    line_chart.render_to_file(chart_file)

    return jsonify({"message": "Graph generated", "file": chart_file})

# Iniciar monitoreo en un hilo
import threading
thread = threading.Thread(target=monitor_interface, daemon=True)
thread.start()

# Iniciar Flask
if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)

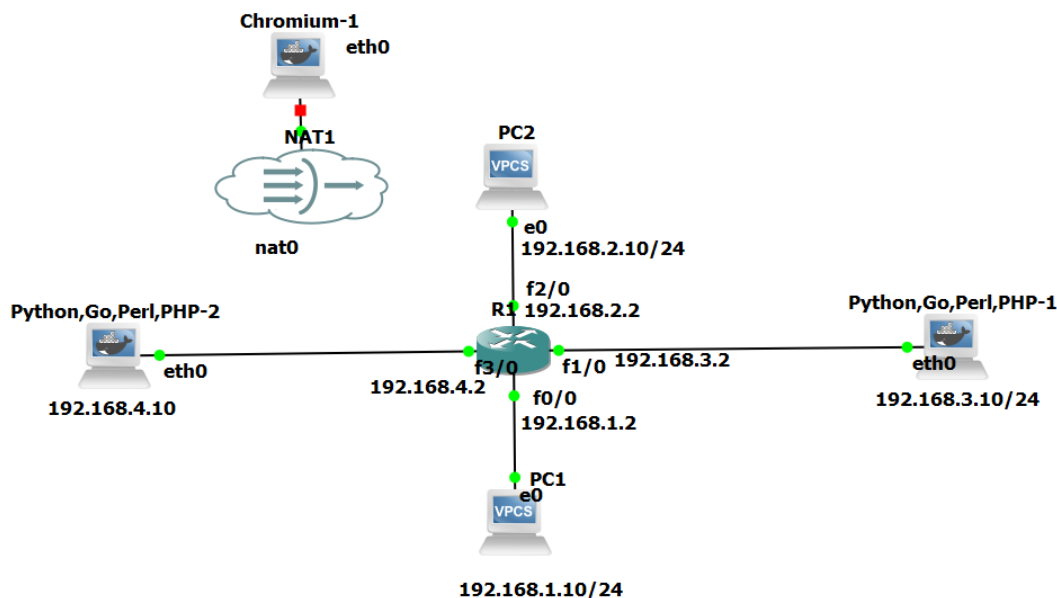
root@Python,Go,Perl,PHP-1:~#
```



2. TOPOLOGÍA

Instituto Politécnico Nacional

Escuela Superior de Cómputo
ADMINISTRACIÓN DE SERVICIOS EN RED



```

root@Python,Go,Perl,PHP-1:~# python3.8 bueno.py
* Serving Flask app 'bueno'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://192.168.3.10:5000
Press CTRL+C to quit
192.168.4.10 - - [20/Dec/2024 13:33:08] "POST /monitor HTTP/1.1" 200 -
[2024-12-20 13:33:10.016401] Packets: 9
[2024-12-20 13:33:31.076194] Packets: 9
[2024-12-20 13:33:52.134935] Packets: 9
[2024-12-20 13:34:13.194966] Packets: 9
  
```

```

root@Python,Go,Perl,PHP-2:~# nano im.py
root@Python,Go,Perl,PHP-2:~# curl -X POST http://192.168.3.10:5000/monitor
{"message":"Monitoring started"}
root@Python,Go,Perl,PHP-2:~#
  
```

